

30-Day FreeRTOS Course for ESP32 Using ESP-IDF

(Day 26)

**“Real-Time Debugging
with FreeRTOS Trace”**



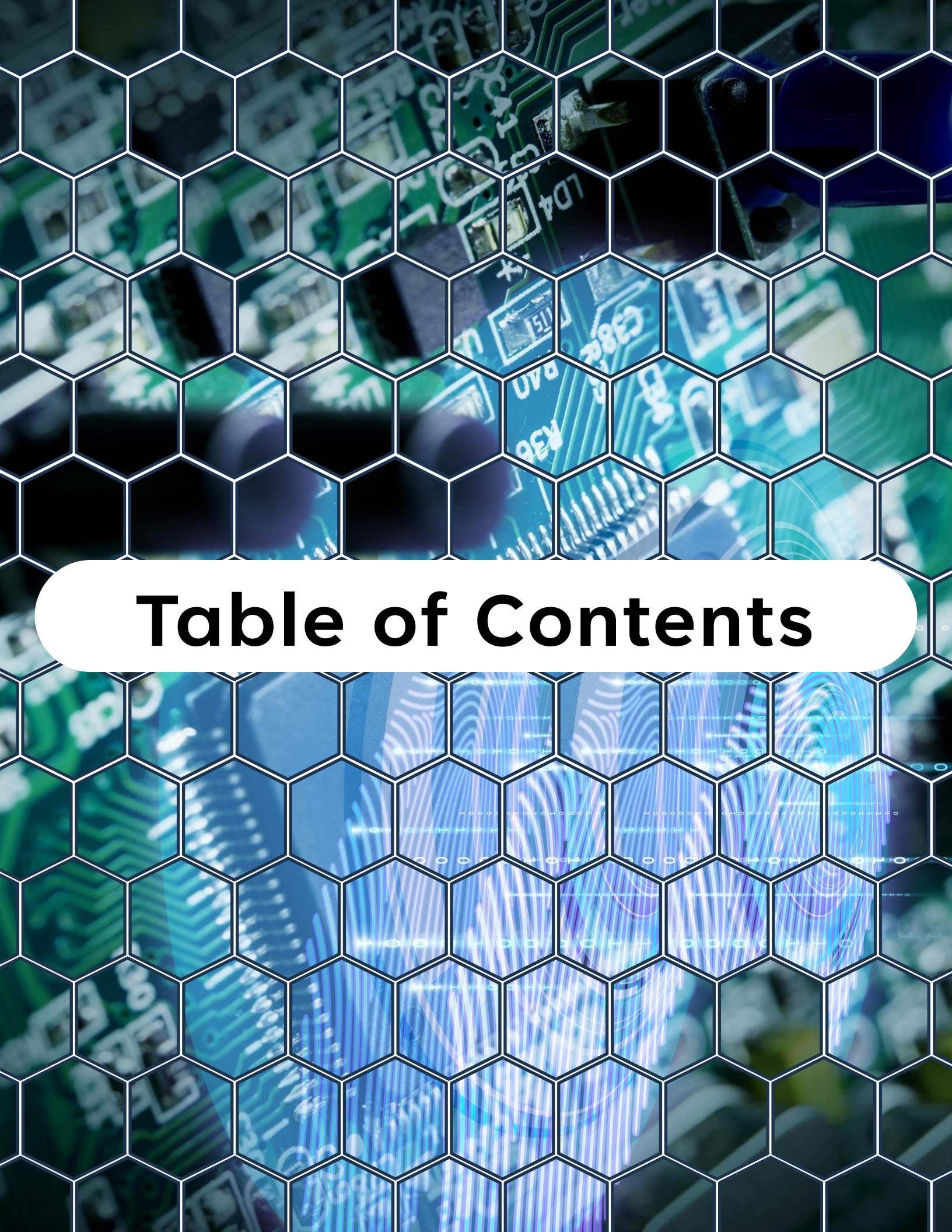
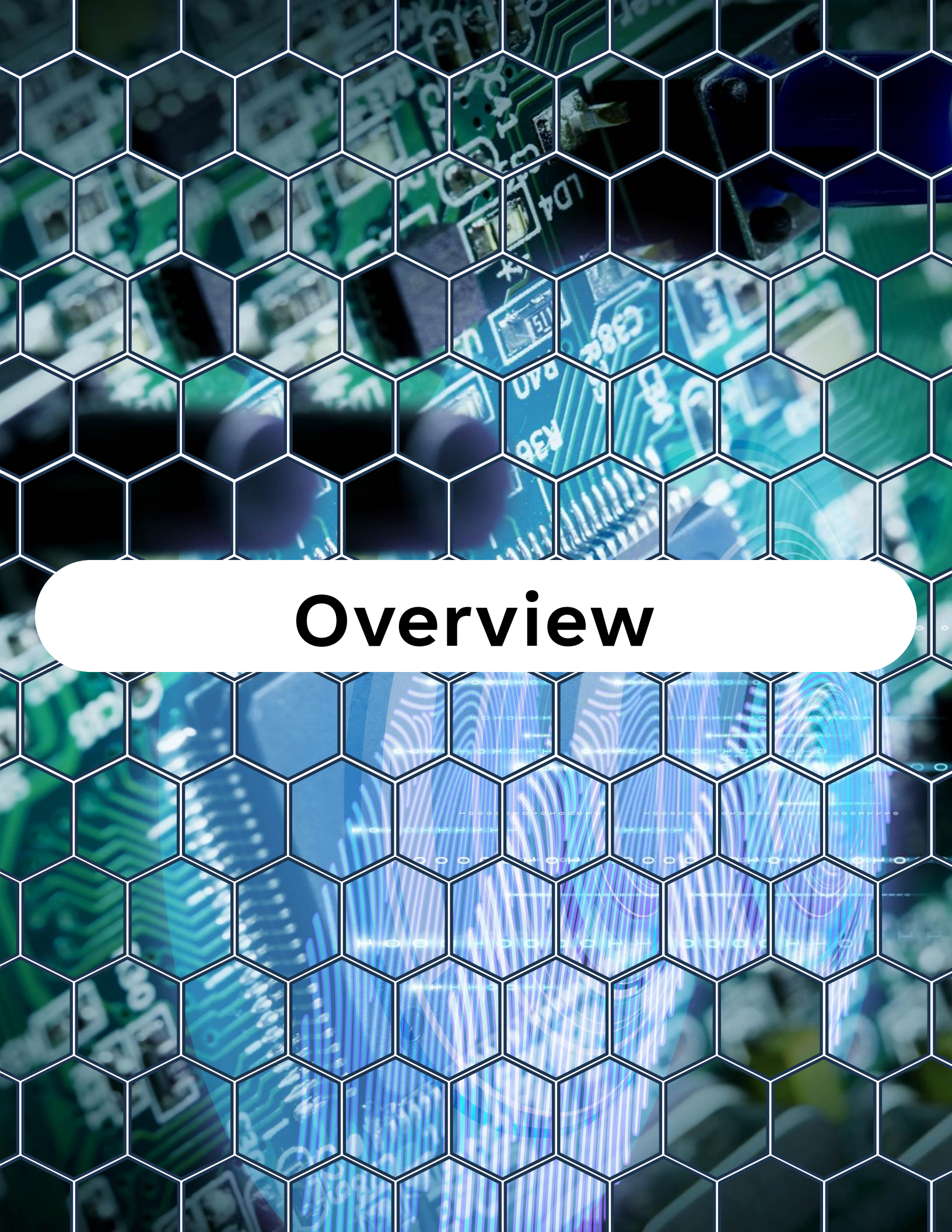


Table of Contents

Table of Contents

1. Overview
2. Why Real-Time Debugging?
3. FreeRTOS Trace Support
4. Tools for Real-Time Debugging
5. Example – FreeRTOS Runtime Stats
6. Best Practices
7. Summary
8. Challenge for Today



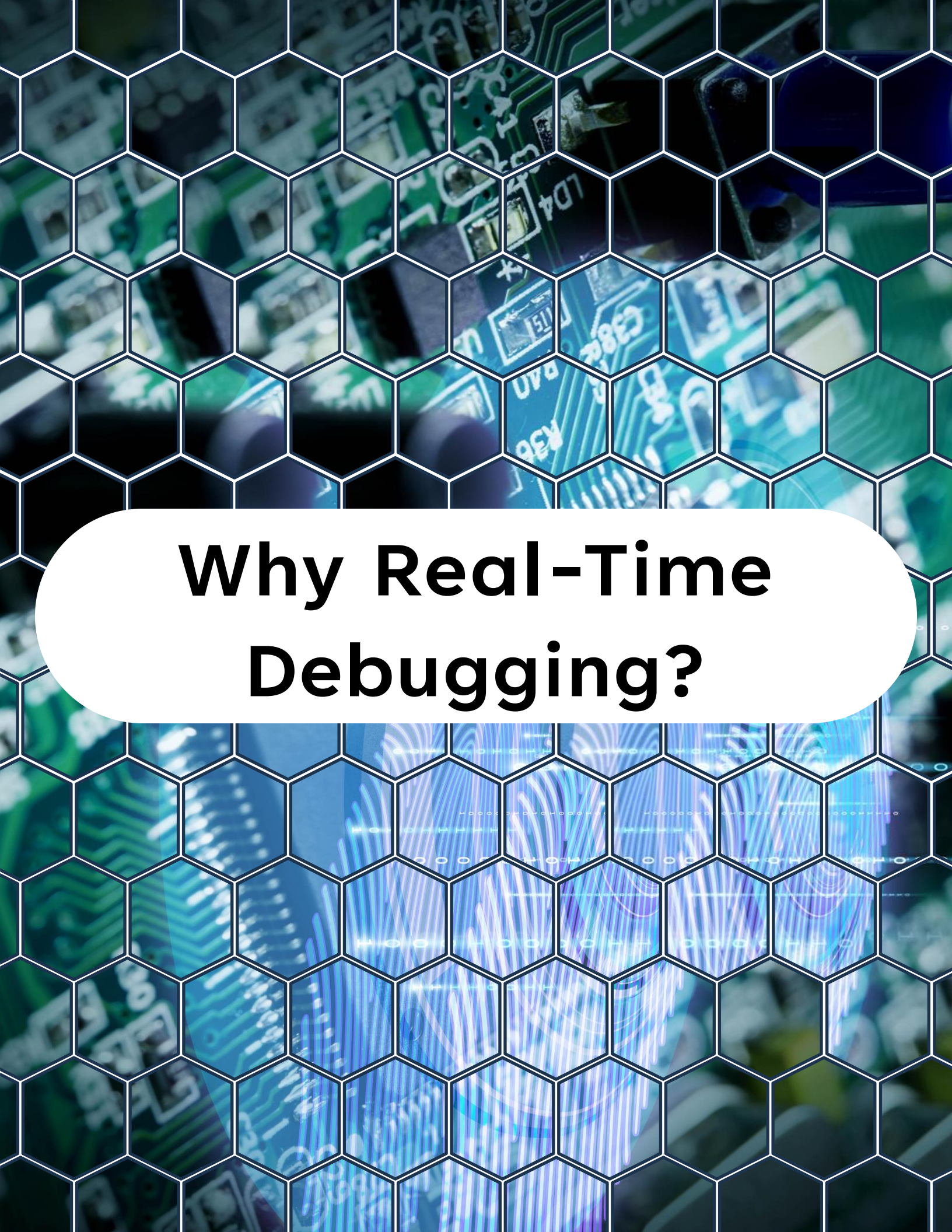
Overview

Overview

On **Day 26**, you'll learn:

- Why **real-time debugging** is essential in RTOS-based systems
- The debugging challenges in FreeRTOS applications
- What **FreeRTOS trace macros** are and how they work
- Tools available for ESP32 FreeRTOS debugging (Tracealyzer, Segger SystemView, runtime stats)
- A practical example: enabling and collecting FreeRTOS runtime trace data
- Best practices for analyzing real-time behavior

By the end of this lesson, you'll understand how to **monitor, visualize, and debug FreeRTOS tasks and communication primitives in real time.**



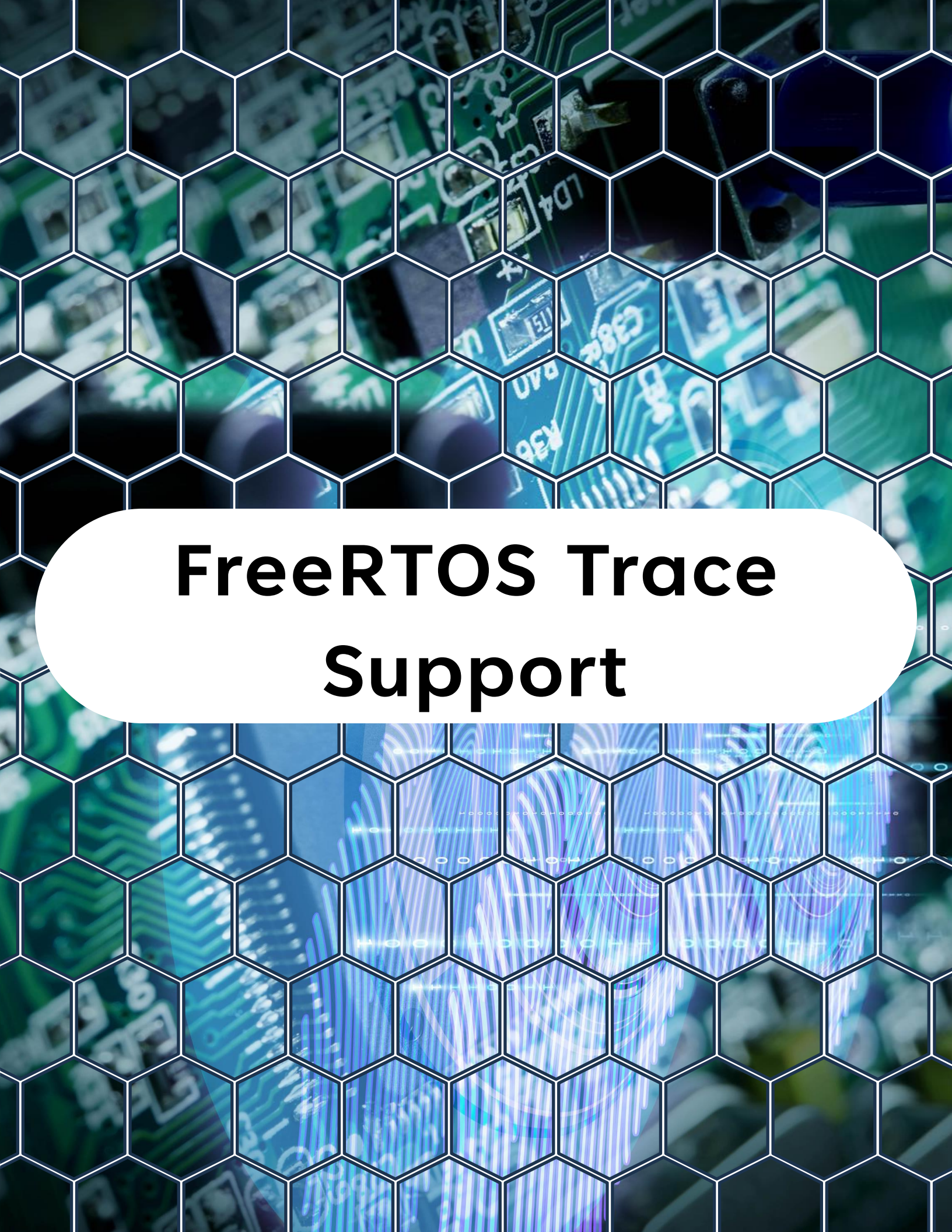
Why Real-Time Debugging?

Why Real-Time Debugging?

Traditional debugging (step-by-step breakpoints) doesn't work well in RTOS systems:

- **FreeRTOS is time-sensitive:** breaking execution changes scheduling behavior.
- **Tasks and ISRs interact concurrently:** race conditions may not appear under stop-and-inspect debugging.
- **Hard-to-reproduce bugs:** deadlocks, priority inversion, starvation.

 Real-time debugging gives insight into **what tasks are running, when they switch, and how they communicate** without stopping the system.



FreeRTOS Trace Support

FreeRTOS Trace Support

FreeRTOS includes trace macros that instrument the kernel. These capture:

- **Task creation, deletion, switching**
- **Queue, semaphore, event group operations**
- **ISR entry/exit**

Trace data can be collected and visualized in external tools.



Tools for Real-Time Debugging

Tools for Real-Time Debugging

1. Runtime Stats (Built-in)

- Enable in menuconfig:

Component config → FreeRTOS → Enable
FreeRTOS runtime stats

- Use APIs like `vTaskGetRunTimeStats()` to print CPU usage by task.

2. Percepio Tracealyzer

- A commercial tool that visualizes FreeRTOS traces.
- Shows task execution timelines, CPU usage, and synchronization events.
- Requires enabling **FreeRTOS trace macros** and sending data via SWO, UART, or buffer dumps.

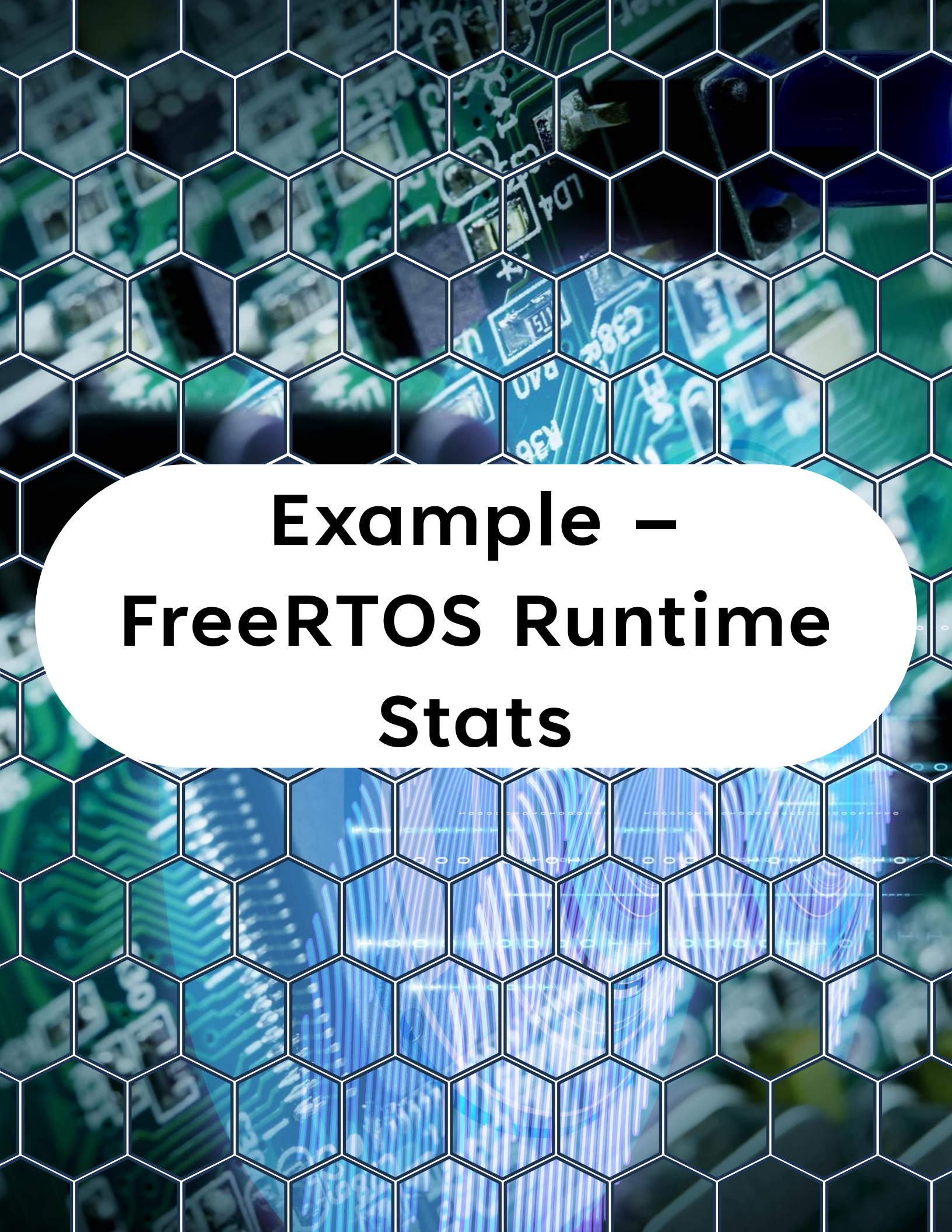
Tools for Real-Time Debugging

3. Segger SystemView

- A free alternative that also visualizes task switches and interrupts.
- Works with J-Link debuggers and ESP32 trace support.

4. ESP-IDF Monitor & Logging

- `idf.py monitor` shows crash traces and logs.
- Combined with `esp_log` **levels** (ERROR, WARN, INFO, DEBUG) for real-time tracing.



Example – FreeRTOS Runtime Stats

Example – FreeRTOS Runtime Stats

Here's a minimal example showing how to enable runtime statistics and log task CPU usage.

Code Example

```
1
2 #include <stdio.h>
3 #include "freertos/FreeRTOS.h"
4 #include "freertos/task.h"
5 #include "esp_log.h"
6
7 #define TAG "DAY26"
8
9 // Busy task that simulates work but yields so idle/WDT can run
10 static void busy_task(void *pvParameters)
11 {
12     (void)pvParameters;
13     while (1) {
14         // Simulate CPU work
15         for (volatile int i = 0; i < 100000; i++) {
16             // do nothing; just burn some cycles
17         }
18         // **CRITICAL**: yield so IDLE task can run and feed the Task WDT
19         vTaskDelay(1); // 1 tick
20         // Alternatively: taskYIELD(); (less robust than a delay of >=1 tick)
21     }
22 }
23
24 // Idle-ish task that sleeps most of the time
25 static void idle_like_task(void *pvParameters)
26 {
27     (void)pvParameters;
28     while (1) {
29         vTaskDelay(pdMS_TO_TICKS(1000));
```

Example – FreeRTOS Runtime Stats

```
24 // Idle-ish task that sleeps most of the time
25 static void idle_like_task(void *pvParameters)
26 {
27     (void)pvParameters;
28     while (1) {
29         vTaskDelay(pdMS_TO_TICKS(1000));
30     }
31 }
32
33 // Task to periodically print runtime stats
34 static void stats_task(void *pvParameters)
35 {
36     (void)pvParameters;
37     // Big enough for typical task lists (adjust if you have many tasks)
38     char buffer[1024];
39
40     while (1) {
41         // Header makes the output easier to scan
42         ESP_LOGI(TAG, "----- Runtime Stats (time, %% CPU) -----");
43         vTaskGetRunTimeStats(buffer);
44         ESP_LOGI(TAG, "\n%s", buffer);
45         vTaskDelay(pdMS_TO_TICKS(5000));
46     }
47 }
48
49 // Main application entry point
50 void app_main(void)
51 {
52     // Create tasks with different behavior
53     // Priorities are fine; the important part is BusyTask yielding
54     // periodically.
55     xTaskCreate(busy_task, "BusyTask", 2048, NULL, 5, NULL);
56     xTaskCreate(idle_like_task, "IdleTask", 2048, NULL, 4, NULL);
57     xTaskCreate(stats_task, "StatsTask", 4096, NULL, 3, NULL);
58
59     ESP_LOGI(TAG, "Day 26: Runtime stats collection started");
60 }
```

Example – FreeRTOS Runtime Stats

Expected Behavior

- BusyTask consumes most of the CPU.
- IdleTask barely uses any CPU.
- StatsTask prints runtime stats every 5 seconds:

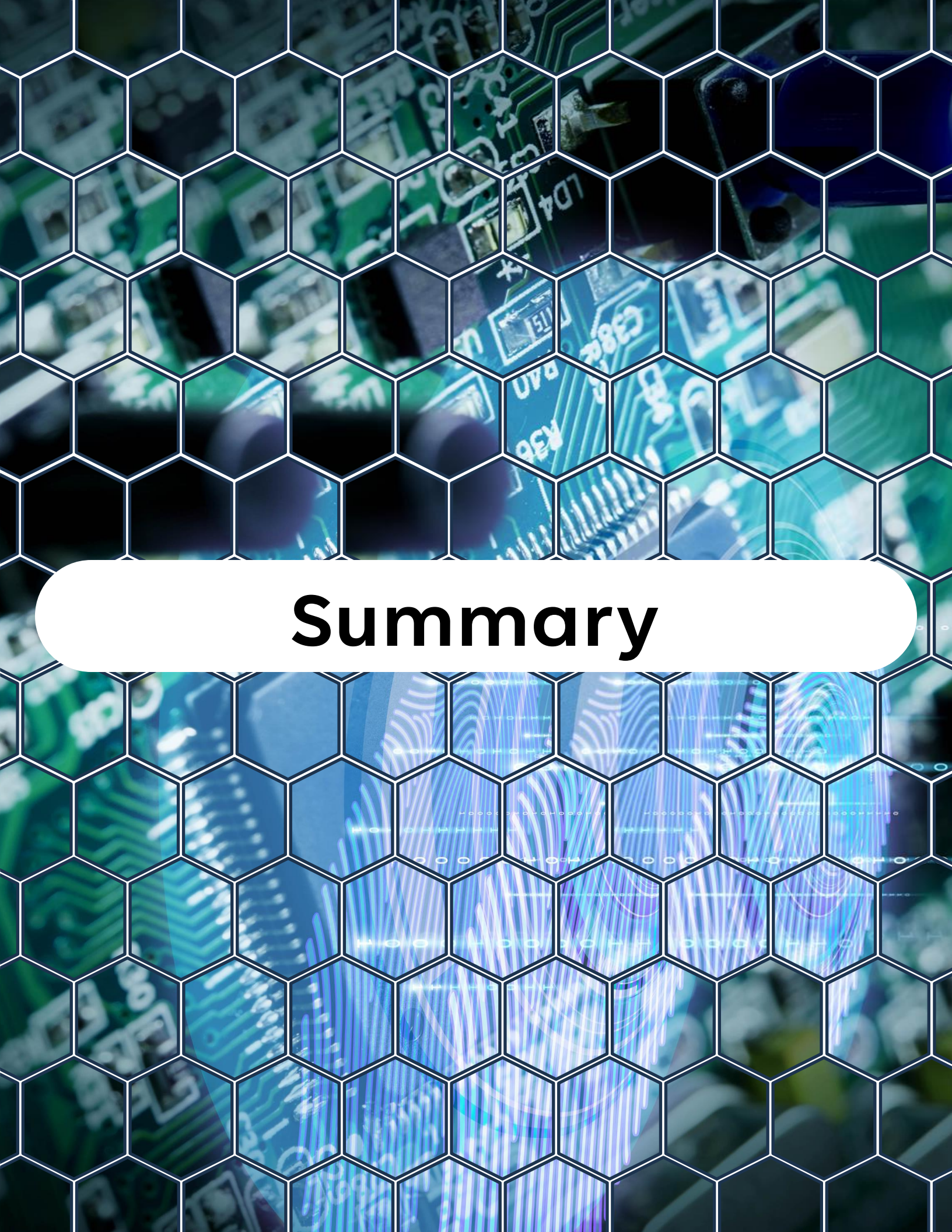
Task Name	Time (ms)	CPU%
BusyTask	4500	90%
IdleTask	200	4%
StatsTask	300	6%



Best Practices

Best Practices

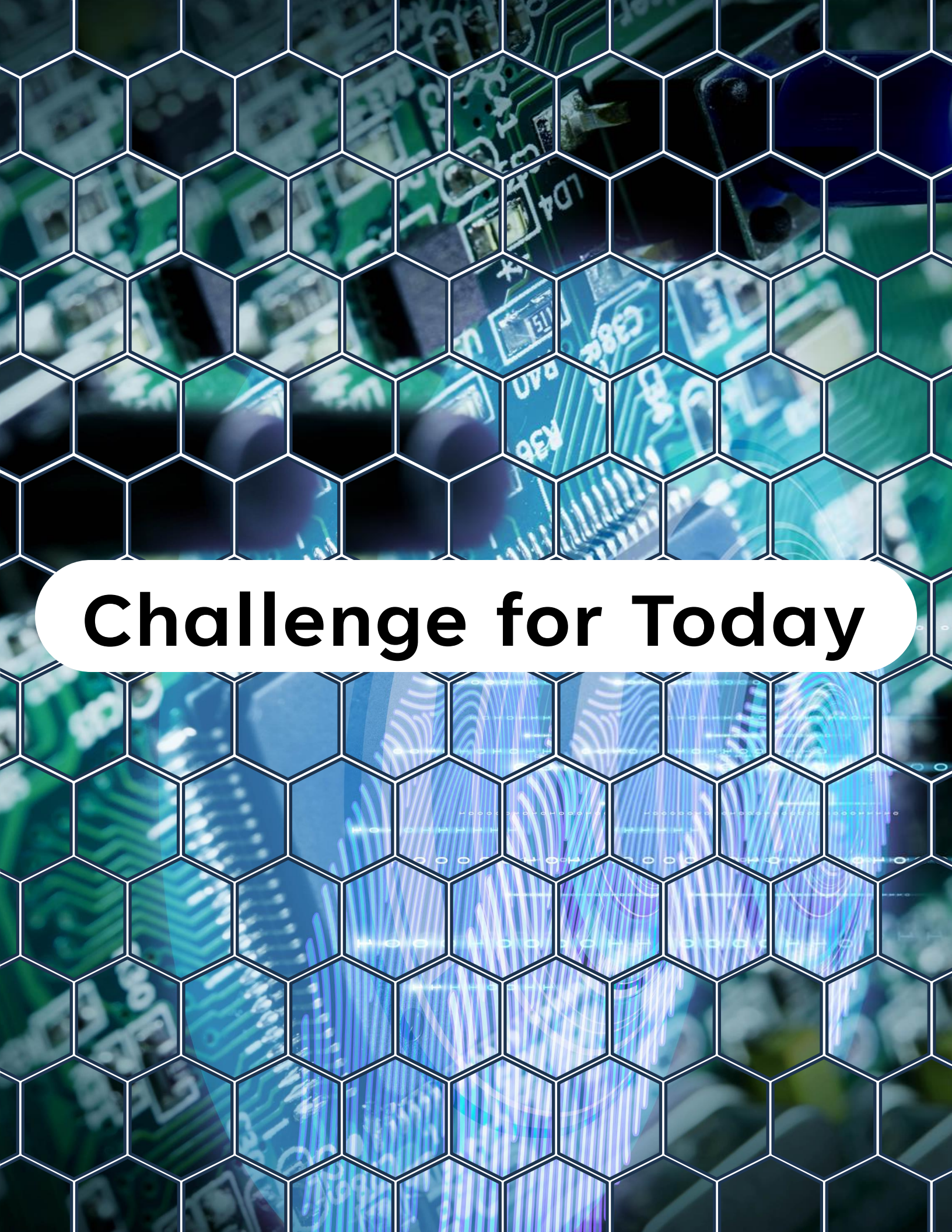
- ✓ Use **runtime stats** during development to tune task priorities and CPU usage.
- ✓ For deeper analysis, use **Tracealyzer** or **SystemView** to visualize scheduling.
- ✓ Minimize logging inside critical paths (UART logging can skew timing).
- ✓ Enable **stack overflow checks** and **heap monitoring** alongside trace.
- ✓ When debugging power or timing issues, combine **tick trace + event trace**.



Summary

Summary

- Real-time debugging is essential for **timing-sensitive RTOS systems**.
- FreeRTOS provides **trace hooks** and runtime stats for monitoring.
- Tools like **Tracealyzer** and **SystemView** provide visual insights.
- ESP-IDF monitor + logging can supplement debugging.
- Always analyze **task timing, communication events, and resource usage** under realistic loads.



Challenge for Today

Challenge for Today

Extend today's project by:

- Add a **queue** between two tasks and use `vTaskGetRunTimeStats()` to see CPU load with communication overhead.
- Experiment with **task priorities** and observe scheduling changes in stats.
- Enable Stack Monitoring
 1. Use `uxTaskGetStackHighWaterMark()` for each task.
 2. Print the high-water mark periodically to detect potential stack overflows.
 3. Experiment with smaller stack sizes and observe when tasks start running into stack issues.



"Thanks for watching!

*If you enjoyed this video, make sure to hit
the like button and subscribe to stay
updated with my latest content.*

*Don't forget to check out my other videos
for more tips and tutorials on Embedded
C, Python, hardware designs, etc. Keep
exploring, keep learning, and I'll see you in
the next video!"*

<https://www.linkedin.com/in/yamil-garcia>
<https://www.youtube.com/@LearningByTutorials>
<https://github.com/god233012yamil>